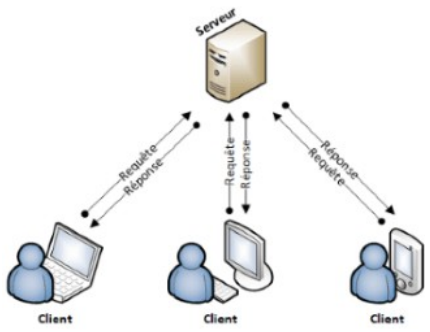
Développement des composants distribués Web

CHAPITRE 1 :INTRODUCTION AUX ARCHITECTURES DISTRIBUÉES

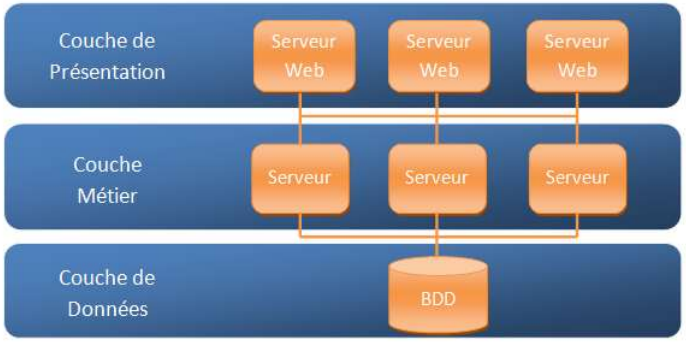
Introduction

- Qu’est ce qu’une architecture logicielle distribuée :
  - Une application à base de composants réparties sur plusieurs machines interconnectés par un réseau, communicant ensemble d’une façon cohérente pour offrir un ensemble de services aux divers clients,
  - Au lieu d'un seul ordinateur puissant, on utilise un réseau d'ordinateurs pour traiter des tâches complexes,



architecture clients/serveur (2 tiers)

Architectures 3 tiers



Architecture n tiers : application distribuée

- Attention : différence entre concurrence, parallélisme et distribution

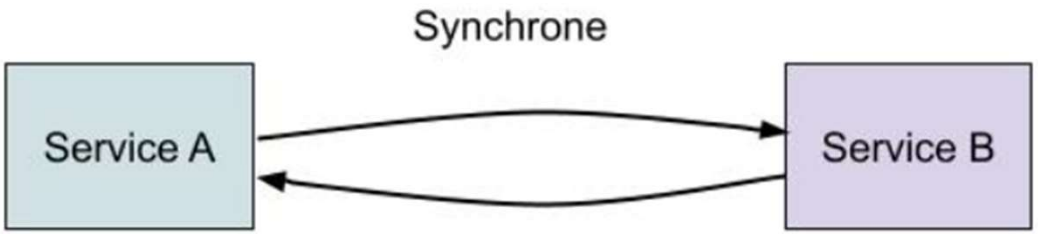
Problématiques

- ✓ L'application ne fonctionne pas sur une machine mais sur plusieurs.
  - ✓ programmation réseau: efficacité, performance, sécurité, tolérance aux pannes, . . .
  - ✓ Approches distribuées : communication, synchronisation, gestion de conflit d'accès …
  - ✓ Comment structurer l'application Architecture logicielle, matérielle et serveurs associés, déploiement
- ✓ L'application utilise/ou réutilise des applications existantes.
  - ✓ Services standards : sécurité, gestion de traces, monitoring
  - ✓ services de connexion à une bases de données : relationnelles, Nosql, clusters big data
    - ✓ Données non persistantes/ données persistantes
  - ✓ Réduction cout de développent
- ✓ Hétérogénéité des plateformes d'exécution : architecture des processeurs, OS et langage de programmation différent : problème d'interopérabilité.
- ✓ Applications Portables

Problématiques

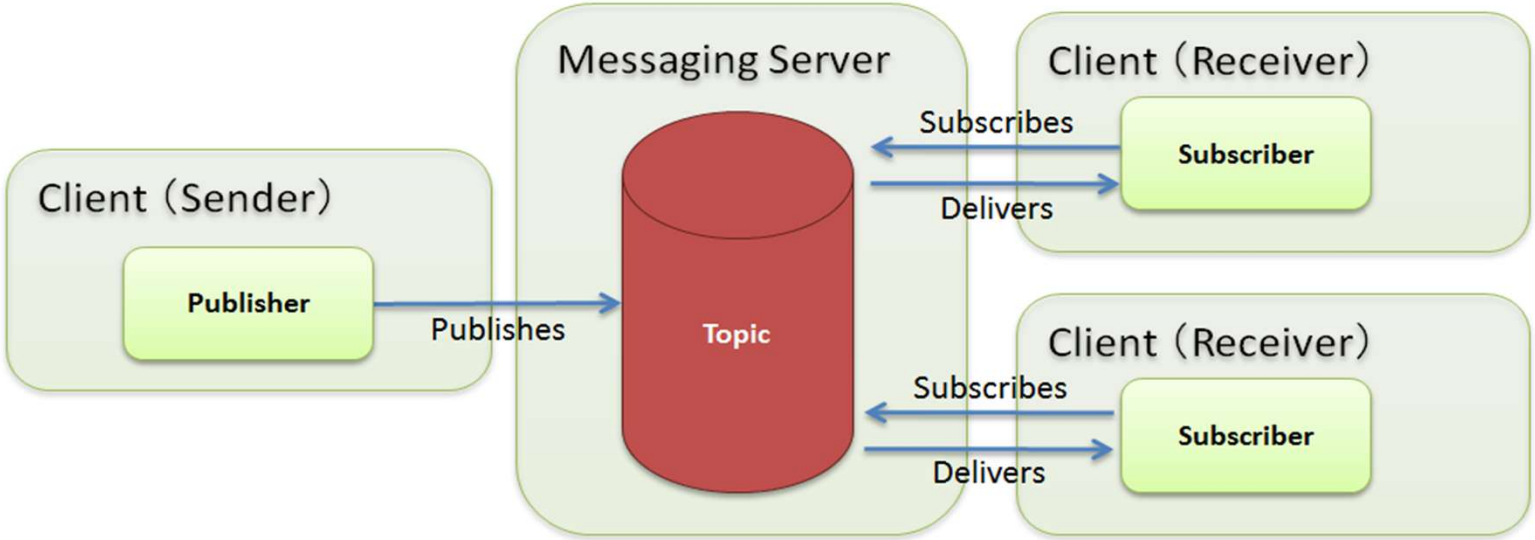
- Espace mémoire diffèrent : problème de passage de messages,
- Communication synchrone / asynchrone
- Comment retrouver un service
- Services disponibles ou pas
- Accès concurrent à un service
- Application scalable : supporte la montée en charge d'utilisation
- Besoins d'abstraction
  - Séparation de l'application en couches
- Besoins de normalisation
  - Applications intégrables et extensibles

Communication synchrone



- permet d'échanger des données de manière bloquante directement à travers un protocole de communication
- Données envoyées en temps réel
- Classiquement RMI, IIOP, nouvellement Resttemplate, Springcloud feign
- Exemple typique Service de recherche d'une information

Communication asynchrone



- permet d'échanger des données de manière non bloquante en utilisant un courtier de messages (broker)
- Application scalable => Favoriser la communication asynchrone
- Classiquement JMS, nouvellement KAFKA, RabbitMQ
- Exemple typique Service tableaux de bord

Problème de communication : intergiciel ou middleware

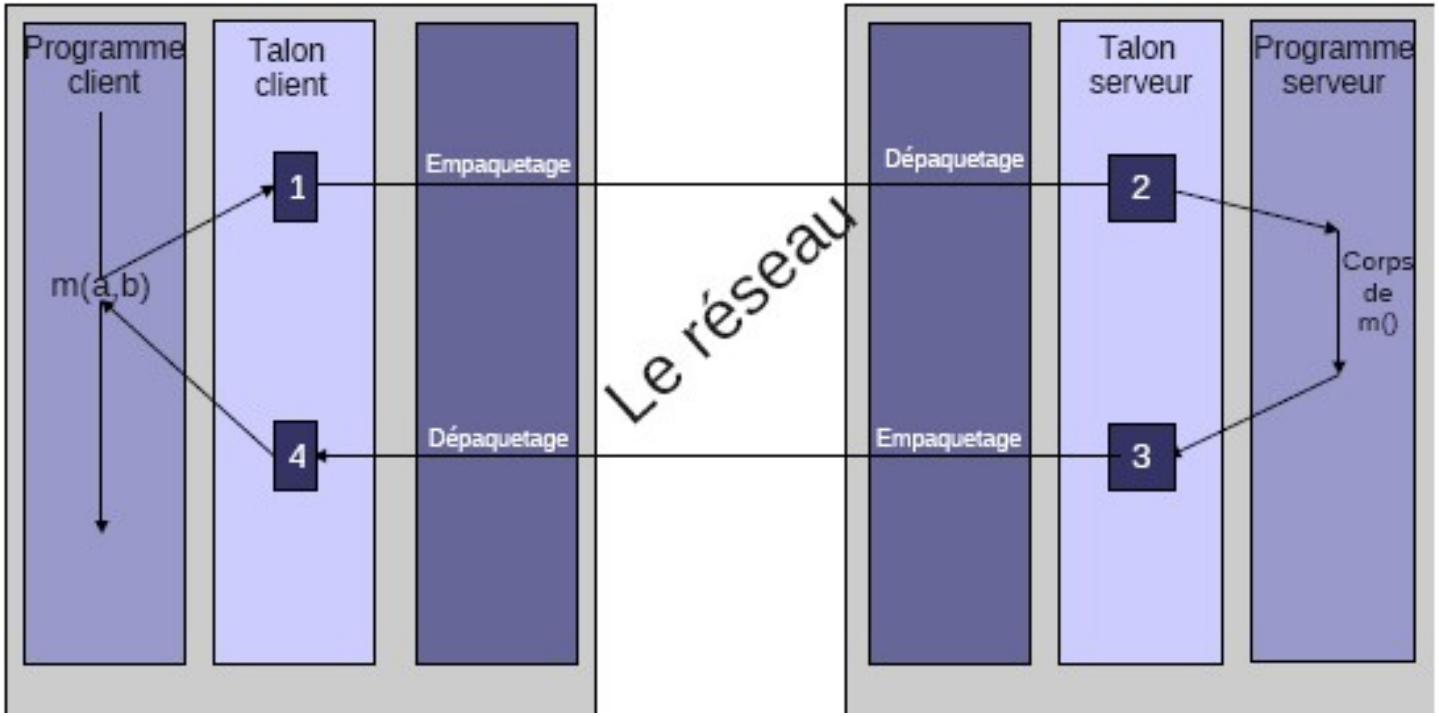
Un **intergiciel** (middleware) : programme qui s'insère entre deux applications assurant la communication des machines selon un protocole bien déterminé.

- ✓ permet à des composants hétérogènes de communiquer et d'échanger des données de manière transparente,
- ✓ synchronisation entre le client et le service,

Problème d’interopérabilité

Communication synchrone entre services : invocation de service à distance

Talon : assurer la communication synchrone entre le client et le fournisseur de service,
Assure l’interopérabilité entre le client et le serveur
Talons = amorces= mandataires=objet proxy=souches=stub



Problème d’interopérabilité

Talon coté client:

- reçoit l’appel en mode local.
- Emballe les paramètres et le transforme en appel distant en envoyant un message.
- attend les résultats en provenance du serveur.
- Déballe les paramètres résultats, et les retourne au programme client comme dans un retour de procédure local.

Problème d’interopérabilité

Talon coté serveur:

- Reçoit l’appel sous forme de message.
- Déballe les paramètres d'appel.
- Lance l’exécution sur le site serveur de la procédure du service invoqué.
- Emballe les paramètres résultats et les retransmet par message.

Problème de passage de paramètres

- classiquement la sérialisation : passage de paramètres (objet) codés en binaires
  - Plus rapide : performance temps réel
  - Peu interopérable
- Nouvellement XML/JSON
  - Moins rapide
  - Très interopérable
- Pas de passage de références

Tolérances aux pannes

- Tolérance aux pannes réactive
  - correction après la panne
- Tolérance aux pannes proactive
  - anticiper et prévenir les défaillances avant qu'elles n'impactent le système.
  - Tache de monitoring du QOS des services,
- Un problème réseau rend l’application inutilisable.
  - Panne coté serveur : attente indéfinie par le client d’une réponse qui ne viendra peut être jamais.
    - Utilisation d’une horloge de garde.
    - Notion de timeout
    - Risque de confusion après relance du client entre les nouvelles réponses attendues et les réponses de l’ancienne requête.
      - Distinction entre requête utiles et orphelines (à détruire).

Comment retrouver un service

- Annuaire de service ou service de nommage
- un système qui associe des noms compréhensibles par les humains à des informations techniques capables de localiser un service
  - Comparable à un DNS
- une couche d'abstraction et de portabilité pour la découverte de services dans des environnements distribués.

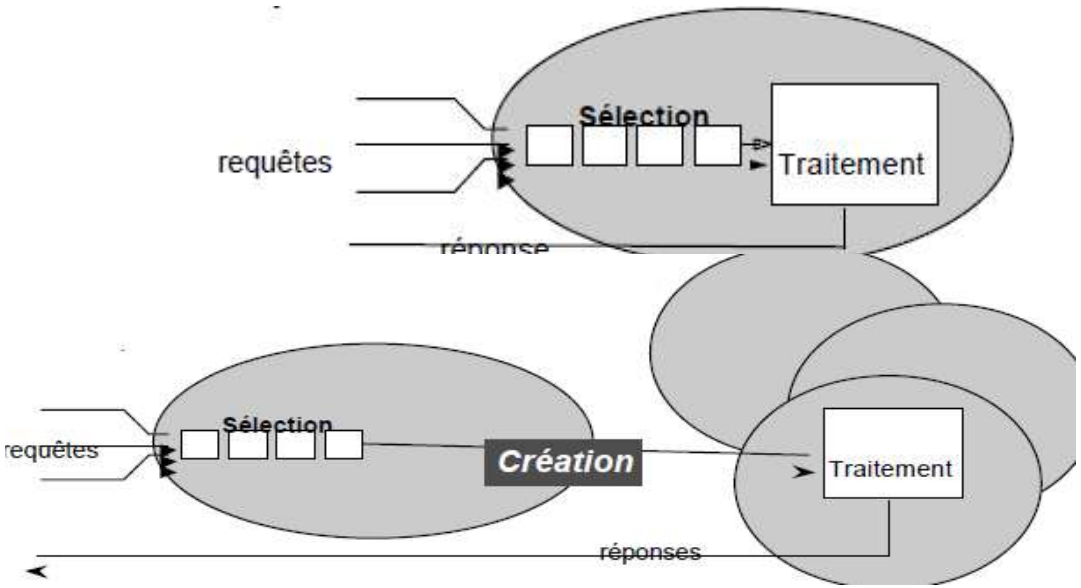
Scalabilité

La scalabilité:

Applications supportant la montée en charge.

Processus serveur unique

Processus par service



- Scalabilité horizontale : ajouter des serveurs au cluster
- Scalabilité verticale : augmentation des performances d'un système en ajoutant des ressources (CPU, RAM, stockage)

Architecture logicielle

- Architecture Logicielle
  - Elle structure et décompose de façon logique chaque application en couches
  - Elle introduit les notions et concepts de découpage en couches, modules, composants, patrons de conception (design patterns) regroupés dans des API gérés par des frameworks
    - Découpage interne du bloc applicatif en couches motifs de conception (Design Patterns)
      - Un patron de conception est une solution générale et réutilisable d'un problème récurrent,
      - Formalisation de bonnes pratiques
    - Framework (« cadre de travail ») et services techniques (compilation, gestion des transactions, logs, traces, gestion des fichiers de configuration...) telque springMVC, SpringBoot

MVC : Modèle Vue contrôleur

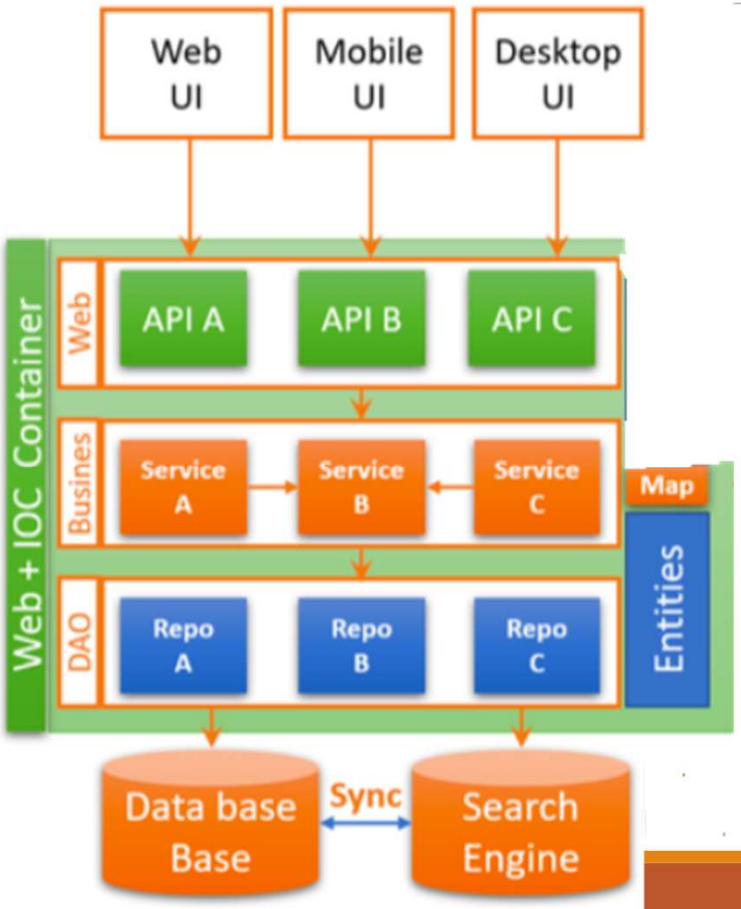
- L'architecture **MVC (Modèle-Vue-Contrôleur)** est un motif de conception logicielle utilisé pour séparer l'interface utilisateur de la logique métier.
- **Modèle (Model) :** Gère les données et la logique métier. Il communique avec la base de données et définit les règles de gestion (ex: CRUD, validation de données).
- **Vue (View) :** Gère l'affichage et l'interface utilisateur. Elle présente les données fournies par le modèle de façon visuelle.
  - Client lourd : interface desktop
  - Client léger : client web
  - Client riche : un compromis entre le client léger et le client lourd, exemple Recat.js, Flutter/React Native (multi-plateformes web/mobile)
- **Contrôleur (Controller) :** Agit comme un intermédiaire. Il reçoit les entrées de l'utilisateur (clics, formulaires), demande les données au Modèle et choisit la Vue à afficher.
- Vue = la partie frontend, contrôleur + modèle= la partie backend.

Framework MVC

- **PHP :** Laravel, Symfony.
- **Python :** Django (variante MVT).
  - Dans MVC classique, le Contrôleur reçoit la requête, manipule le Modèle, puis choisit la Vue à afficher. Dans MVT, le framework (comme Django) gère souvent cette sélection de Template
  - **Modèle (Model)** : Identique à MVC,
  - **Vue (View)** (dans MVT) : C'est la partie qui reçoit les requêtes, interagit avec le Modèle pour récupérer les données, puis passe ces données au Template pour l'affichage. C'est l'équivalent du **Contrôleur** dans MVC.
  - **Template (Template)** (dans MVT) : C'est le fichier qui contient le code d'affichage (HTML, CSS, etc.), qui reçoit les données préparées par la Vue (Contrôleur) et les présente à l'utilisateur. C'est l'équivalent de la **Vue** dans MVC.
- **Java (backend):** Spring MVC.
- **JavaScript (frontend) :** Angular, React.js

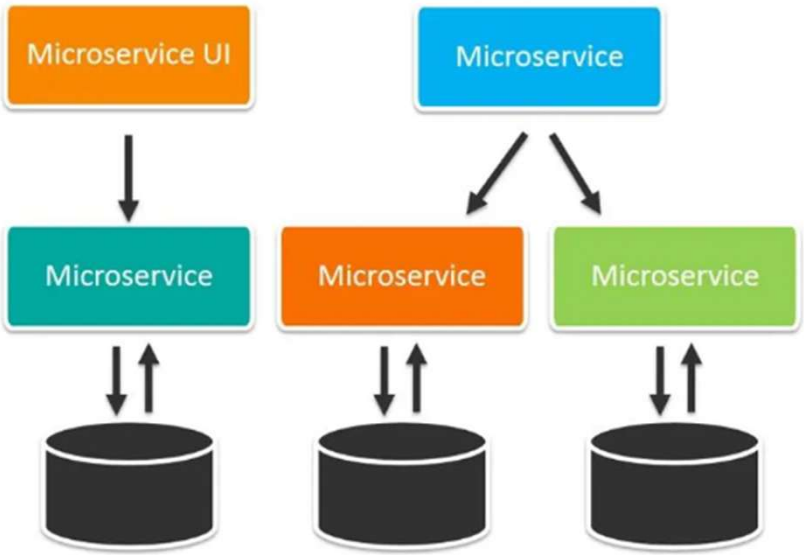
Architecture monolithique

- Application web via API web
- API interopérable avec plusieurs types de client
- Couche web/ métier/ accès aux données
- Communication synchrone/asynchrone entre services
- Déploiement de toutes les couches sur un serveur
- Stockage de données sur des bases de données relationnelle/NoSQL
- Stockage de données sur des "search Engine" big data facilitant la fouille et l'archivage de données comme ElasticSearch ou Hadoop



Architecture micro-services

- Bases de données réparties/micro-services
- Communication synchrone/asynchrone entre microservices
- Communication client/microservices
- déploiement sur cloud
- Point d'accès unique : gateway
- Équilibrage de charge (load balancing) entre micro-services



Technologies utilisées

- Architecture orientée objets distribués (JEE: Jakarta Entreprise Edition) : basé sur RPC (Remote Procedure Call)
  - API à utiliser RMI (Remote Method Invocation): communication synchrone par invocation de méthodes à distance, échange de message par sérialisation (forme binaire)
  - JMS (Java Messaging Service): communication asynchrone par échange de message
  - CORBA (Common Object Request Broker Architecture) (technologie JEE) : communication synchrone par invocation de méthodes à distance entre objets hétérogène (JAVA/C++)
- Architecture orientée composant EJB(Entreprise Java Beans) : composants (Beans) distribué déployés sur divers serveurs d'application
  - Serveur d'application : tomcat, wildfly, Glassfish, …
  - Largement influencé par RMI.
- Architecture orientée Service (SOA) : communication synchrone à travers SOAP (Simple Object Access Protocol) basé sur des messages XML échangés client/service ou service/service.
- Architecture monolithique (API Restful Framework SpringMVC): API web communicant d'une façon synchrone/ asynchrone sur le HTTP basée sur des messages HTML, XML ou JSON entre client /API web ou API web/API web,
- Architecture micro-services (API Restful Framework Springboot) : micro services déployés sur un cloud (cluster de machines serveurs)

Créer des API web : autres solutions

- REST est un style architectural largement utilisé, basé sur les méthodes HTTP pour manipuler des ressources.
  - REST est un style architectural pour concevoir des APIs en utilisant les méthodes HTTP (GET, POST, PUT, DELETE) pour interagir avec des ressources identifiées par des URLs.
  - Inconvénients : le problème N+1 requêtes, exemple la recherche des livres d'auteurs de la région de Sfax,
- GraphQL est un langage requête pour les API qui permet aux clients de demander des données spécifiques à partir d'un seul point de terminaison.
  - Plus efficace que REST, car il évite le sur-fetch et sous-fetch. flexible pour les clients qui ont besoin de données spécifiques. =>plus performant pour certaines applications,
  - La réponse d'une API GraphQL est un objet JSON qui peut contenir les clés principales suivantes :
    - data : Cette clé contient les résultats de l'opération exécutée. Les données retournées correspondent exactement aux champs que le client a spécifiés dans sa requête.
    - errors : Cette clé contient un ensemble d'erreurs produites lors de l'exécution de la requête.
    - **Limiter le nombre de requêtes, Simplifier le développement front-end** (Les développeurs front-end peuvent obtenir les données dont ils ont besoin sans devoir attendre des changements au niveau du back-end).
  - Plus complexe à mettre en œuvre que REST,

Créer des API web : autres solutions

- gRPC est un framework RPC (Remote Procedure Call) performant, souvent utilisé pour la communication entre services.
  - utilise protobuf pour la sérialisation des données qui offre une communication bidirectionnelle (streaming).
  - Protobuf: langage de description d'interface et format de sérialisation de données structurées (énumérations, liste, …: codées en divers langage de programmation c++, java, python, …) développé par Google, offrant un format plus petit, plus rapide et plus simple que des formats comme XML ou JSON, pour la communication entre services et le stockage de données
  - Un compilateur (protoc) génère automatiquement le code client et serveur dans différents langages à partir d'un fichier de définition
  - Performances élevées, plus performant que REST dans les scénarios où la vitesse est cruciale.
  - Moins flexible que REST et GraphQL

- Websocket est une API + un protocole (ws qui remplace le http) pour établir une connexion bidirectionnelle, persistante et en temps réel entre un client (navigateur web par exemple) et un serveur,
  - Actions initiées par les deux parties client ou serveur
  - Connexion persistante => performance
  - Faible surcharge de données: minimiser les en-têtes de données, ce qui réduit la quantité de données transférées et améliore l'efficacité de la communication